

Doc. no.:	P3961R1
Date:	2026-3-23
Audience:	LEWG
Reply-to:	Zhihao Yuan <zy AT miator.net>

Less double indirection in `function_ref` (RU-220)

- Less double indirection in `function_ref` (RU-220)
 - Introduction
 - Motivation
 - Discussion
 - Why stop at requiring non-noexcept to be compatible with noexcept?
 - Why allow whether `function_ref` referencing another `function_ref` to be unspecified?
 - Why stop at `function_ref` as opposed to extending to `reference_wrapper`?
 - Implementation
 - Wording
 - Acknowledgements
 - References

Introduction

RU-220 calls for allowing skipping indirections when constructing `function_ref` to improve codegen. The proposed change in LWG 4264^[1] is not strictly an optimization because certain behaviors with and without the change are visible, therefore, need to be made unspecified. This paper (P3961) suggests that, in addition to the change, a subset of the "optimized" cases should be mandated.

Motivation

Closely matching the behavior of `noexcept` function types in the core language

Replacing `auto` with `function_ref` CTAD, the assignment behaviors between `noexcept` and non- `noexcept` function objects should stay the same.

Core language	Library
<pre> void foo(int); void bar(int) noexcept; auto p1 = foo; auto p2 = bar; p1 = p2; // ok p1(3); // call bar p2 = p1; // error </pre>	<pre> void foo(int); void bar(int) noexcept; std::function_ref r1 = foo; std::function_ref r2 = bar; r1 = r2; // ok r1(3); // call bar r2 = r1; // error </pre>

Without this paper, `r1 = r2` doesn't compile.

Substituting the `R(Arg)` with the same types into `R(*) (Arg) noexcept(v)` and `function_ref<R(Arg) noexcept(v)>`, the conversion behaviors should stay the same.

Core language	Library
<pre> void foo(int) noexcept; void bar(int) noexcept; void (*p1)(int) noexcept = foo; void (*p2)(int) = p1; p1 = bar; p2(3); // call foo </pre>	<pre> void foo(int) noexcept; void bar(int) noexcept; std::function_ref<void(int) noexcept> r1 = foo; std::function_ref<void(int)> r2 = r1; r1 = bar; r2(3); // call bar </pre>

Without this paper, `r2(3)` calls `foo`.

Discussion

Why stop at requiring non- `noexcept` to be compatible with `noexcept` ?

Although a pointer to non-const member function cannot store a value of a pointer to const member function in the core language, the interactions between the `noexcept` and non-`noexcept` could be generalized to cover `function_ref` with const and non-const signatures.

R1 of the paper now incorporates the change.

Why allow whether `function_ref` referencing another `function_ref` to be unspecified?

By doing so, we can decide whether to mandate another "optimization" in this category in the future without introducing a breaking change on paper.

Although the difference in behaviors is visible, `function_ref` models after rebindable references, and reference shall not reference reference. The users shouldn't expect `function_ref` to exhibit a "reference to reference" behavior. The attack vector of this behavior is also limited. `function_ref` cannot rebind from another `function_ref` of a different signature thanks to its restricted assignment operator.

Why stop at `function_ref` as opposed to extending to `reference_wrapper` ?

The author and the contributor(s) of this paper are welcome to see such a change. However, because unwrapping `reference_wrapper` is a discussed matter about 8 years ago, the author wants to see more evidence in real-world code to justify this change.

Note that *without* unwrapping `reference_wrapper` , a user can call a non-const member `operator()` from a `function_ref` with a const signature: ^{Compiler} `eP11Kh6es` ^{Explorer} (<https://godbolt.org/z/eP11Kh6es>). `move_only_function` has the same problem.

Implementation

The paper has been implemented in `zhihaoy/nontype_functional@p3961r1` (https://github.com/zhihaoy/nontype_functional/tree/p3961r1).

Wording

The wording is relative to N5032.

Modify 22.10.17.1 [func.wrap.general] (<https://eel.is/c++draft/func.wrap.general>) as indicated:

Subclause [func.wrap] (<https://eel.is/c++draft/func.wrap>) describes polymorphic wrapper classes that encapsulate arbitrary callable objects.

Let `t` be an object of a type that is a specialization of `function` , `copyable_function` , `move_only_function` , or `function_ref` , such that the target object `x` of `t` has a type that is a specialization of `function` , `copyable_function` , `move_only_function` , or `function_ref` . Each argument of the invocation of `x` evaluated as part of the invocation of `t` may alias an argument in the same position in the invocation of `t` that has the same type, even if the corresponding parameter is not of reference type.

Modify [func.wrap.ref.ctor] (<https://eel.is/c++draft/func.wrap.ref.ctor>) as follows:

```
template<class... T>
    static constexpr bool is-invocable-using = see below;
```

If *noex* is true, [...]

```
template<class F>
    static constexpr bool is-convertible-from-specialization = see below;
```

If *F* denotes a specialization function `ref<R(Args...) cv2 noexcept(noex2)>` for some placeholders *cv2* and *noex2*, *is-convertible-from-specialization*<F> is equal to:

- *is convertible* `v<R(&)(Args...) noexcept(noex2), R(&)(Args...) noexcept(noex)>`, and
- *is convertible* `v<int cv&, int cv2&>`.

Otherwise, *is-convertible-from-specialization*<F> is false.

[...]

```
template<class F> constexpr function_ref(F&& f) noexcept;
```

Let *T* be `remove_reference_t<F>`.

Constraints:

- `remove_cvref_t<F>` is not the same type as `function_ref`,
- `is_member_pointer_v<T>` is false, and
- *is-invocable-using*<cv *T*&> is true.

Effects: If *is-convertible-from-specialization*<remove_cv t<T>> is false, it initializes *bound-entity* with `addressof(f)`, and *thunk_ptr* with the address of a function *thunk* such that `thunk(bound-entity, call-args...)` is expression-equivalent to `invoke_r<R>(static_cast<cv T&>(f), call-args...)`. Otherwise, initializes *bound-entity* with the value of *f.bound-entity* and *thunk_ptr* with the value of *f.thunk_ptr*.

Remarks: If `remove_cvref t<F>` is a specialization of `function_ref`, an implementation may initialize *bound-entity* with the value of *f.bound-entity* and *thunk_ptr* with the value of *f.thunk_ptr*. [Example:

```

void f1(std::string);
void f2(std::string);

function_ref<void(std::string)> r1(&f1);
function_ref<void(std::string&&)> r2(r1);
r2(""); // f1 is invoked
r1 = &f2;
r2(""); // it is unspecified if f1 or f2 is invoked

```

—end example]

[...]

```

template<class T> function_ref& operator=(T) = delete;

```

Constraints:

- ~~T is not the same type as function_ref~~;
- ~~is_convertible_from_specialization<T> is false~~;
- is_pointer_v<T> is false, and
- T is not a specialization of constant_arg_t.

Update the following macro definition in [version.syn] (<https://eel.is/c++draft/version.syn>):

```

#define __cpp_lib_function_ref 2025112026XXL // freestanding, also in <functi

```

Acknowledgements

Thank Hana Dusíková for bringing up the issue on the reflectors and Tomasz Kamiński for providing implementation experience.

References

1. LWG 4264: *Skipping indirection is not allowed for function_ref*.

<https://cplusplus.github.io/LWG/issue4264> (<https://cplusplus.github.io/LWG/issue4264>) ↩